

CSE 472 Assignment 2: Decision Trees, Random Forests, and Extra Trees

S.M Kausar Parvej
Student ID: 2005076

Department of Computer Science and Engineering
Bangladesh University of Engineering and Technology (BUET)

January 25, 2026

Abstract

This report presents the implementation and evaluation of three tree-based machine learning algorithms: Decision Trees, Random Forests, and Extremely Randomized Trees (Extra Trees). All algorithms were implemented from scratch using only NumPy and evaluated on standard classification datasets (Iris and Wine). The implementations were compared against optimized versions from scikit-learn to validate correctness and analyze performance characteristics. Results demonstrate that ensemble methods significantly improve upon single decision trees, achieving perfect classification on the Wine dataset while maintaining competitive performance on Iris.

1 Introduction

Decision trees are intuitive and powerful machine learning models that create hierarchical decision rules for classification and regression tasks. However, they often suffer from high variance and overfitting, particularly when grown to full depth. Ensemble methods address these limitations by combining multiple trees using different sources of randomness.

This assignment explores three tree-based algorithms:

- **Decision Tree:** A single tree that recursively partitions the feature space
- **Random Forest:** An ensemble using bootstrap sampling and random feature selection
- **Extra Trees:** An ensemble using random threshold selection without bootstrap sampling

The primary objectives are to:

1. Implement these algorithms from scratch without using scikit-learn's tree models

2. Understand the working principles and differences between algorithms
3. Empirically analyze bias-variance tradeoffs
4. Compare custom implementations with industrial-grade libraries

2 Algorithmic Details

2.1 Decision Tree

A decision tree is constructed by recursively splitting the training data based on feature values that maximize information gain. The algorithm uses a greedy approach to select the best split at each node.

2.1.1 Splitting Criteria

Two impurity measures were implemented:

Gini Impurity:

$$\text{Gini}(t) = 1 - \sum_{i=1}^C p_i^2 \quad (1)$$

where p_i is the proportion of class i samples at node t , and C is the number of classes.

Entropy:

$$\text{Entropy}(t) = - \sum_{i=1}^C p_i \log_2(p_i) \quad (2)$$

Information Gain: The quality of a split is measured by information gain:

$$\text{Gain}(S, A, \theta) = \text{Impurity}(S) - \sum_{v \in \{L, R\}} \frac{|S_v|}{|S|} \text{Impurity}(S_v) \quad (3)$$

where S is the parent node, A is the feature, θ is the threshold, and S_L , S_R are left and right child nodes.

2.1.2 Tree Construction Algorithm

Algorithm 1 Build Decision Tree

```
1: function BUILDTREE( $X, y, depth$ )
2:   if stopping criteria met then
3:     return LEAFNODE(most common class in  $y$ )
4:   end if
5:    $(feature, threshold) \leftarrow \text{FINDBESTSPLIT}(X, y)$ 
6:   if no valid split found then
7:     return LEAFNODE(most common class in  $y$ )
8:   end if
9:    $X_L, y_L \leftarrow$  samples where  $X[:, feature] \leq threshold$ 
10:   $X_R, y_R \leftarrow$  samples where  $X[:, feature] > threshold$ 
11:   $left \leftarrow \text{BUILDTREE}(X_L, y_L, depth + 1)$ 
12:   $right \leftarrow \text{BUILDTREE}(X_R, y_R, depth + 1)$ 
13:  return DECISIONNODE( $feature, threshold, left, right$ )
14: end function
```

Stopping Criteria:

- Maximum depth reached
- All samples belong to the same class (pure node)
- Number of samples less than minimum samples per split
- No valid split can be found

2.2 Random Forest

Random Forest is an ensemble learning method that combines multiple decision trees trained on different subsets of data with random feature selection.

2.2.1 Key Differences from Decision Trees

Bootstrap Sampling (Bagging): Each tree is trained on a bootstrap sample of size N drawn with replacement from the original dataset of size N . This introduces diversity among trees.

Random Feature Selection: At each split, only a random subset of m features is considered, where:

$$m = \lfloor \sqrt{d} \rfloor \quad (4)$$

where d is the total number of features. This decorrelates the trees and reduces variance.

Ensemble Prediction: For classification, predictions are made by majority voting:

$$\hat{y} = \text{mode}\{h_1(x), h_2(x), \dots, h_T(x)\} \quad (5)$$

where h_t is the prediction of tree t and T is the number of trees.

2.3 Extra Trees (Extremely Randomized Trees)

Extra Trees introduces additional randomness compared to Random Forests to further reduce variance.

2.3.1 Key Differences from Random Forest

No Bootstrap Sampling: Unlike Random Forest, Extra Trees uses the entire training dataset for each tree. This reduces computation time as bootstrap sampling is not needed.

Random Threshold Selection: Instead of searching for the optimal threshold for each feature, Extra Trees randomly selects a threshold from the feature’s range:

$$\theta \sim \text{Uniform}(\min(X[:, f]), \max(X[:, f])) \quad (6)$$

This additional randomness leads to:

- Faster training (no threshold optimization)
- Lower variance (more diverse trees)
- Potentially higher bias (less optimal splits)

3 Experimental Setup

3.1 Datasets

Two standard classification datasets from scikit-learn were used:

Iris Dataset:

- Samples: 150 (105 training, 45 testing)
- Features: 4 (sepal length, sepal width, petal length, petal width)
- Classes: 3 (Setosa, Versicolor, Virginica)
- Task: Multi-class classification

Wine Dataset:

- Samples: 178 (124 training, 54 testing)
- Features: 13 (chemical properties)
- Classes: 3 (wine cultivars)
- Task: Multi-class classification

Both datasets were split using stratified sampling with a 70-30 train-test ratio to maintain class proportions.

3.2 Hyperparameters

Consistent hyperparameters were used across all models for fair comparison:

Parameter	Value
Maximum Depth	10
Minimum Samples per Split	2
Number of Trees (RF/ET)	100
Max Features per Split	$\sqrt{d}$
Criterion	Gini
Random Seed	76

Table 1: Hyperparameter configuration used in all experiments

3.3 Evaluation Metrics

Three metrics were used to comprehensively evaluate model performance:

Accuracy:

$$\text{Accuracy} = \frac{\text{Number of correct predictions}}{\text{Total number of predictions}} \quad (7)$$

F1-Score (Macro): The harmonic mean of precision and recall, averaged across classes:

$$F1_{\text{macro}} = \frac{1}{C} \sum_{i=1}^C \frac{2 \cdot P_i \cdot R_i}{P_i + R_i} \quad (8)$$

AUROC (Area Under ROC Curve): For multi-class problems, one-vs-rest AUROC with macro averaging was computed.

3.4 Implementation Details

- **Language:** Python 3
- **Libraries:** NumPy (computation), pandas (data handling), scikit-learn (datasets and metrics only)
- **Reproducibility:** Fixed random seed (76) for all experiments
- **Code Structure:** Object-oriented design with separate classes for each algorithm

4 Results

4.1 Overall Performance Comparison

Table 2 presents the complete results for all models on both datasets.

Model	Dataset	Accuracy	F1-Score	AUROC
Custom Decision Tree	Iris	0.8667	0.8667	0.9000
Custom Decision Tree	Wine	0.8889	0.8882	0.9221
Custom Random Forest	Iris	0.9111	0.9118	0.9844
Custom Random Forest	Wine	0.9815	0.9811	0.9998
Custom Extra Trees	Iris	0.9333	0.9333	0.9926
Custom Extra Trees	Wine	0.9815	0.9811	1.0000
Sklearn Decision Tree	Iris	0.8667	0.8667	0.9000
Sklearn Decision Tree	Wine	0.9074	0.9066	0.9346
Sklearn Random Forest	Iris	0.9556	0.9556	0.9926
Sklearn Random Forest	Wine	0.9630	0.9625	1.0000
Sklearn Extra Trees	Iris	0.9333	0.9333	0.9926
Sklearn Extra Trees	Wine	0.9815	0.9811	1.0000

Table 2: Complete performance comparison of all models on both datasets. Bold values indicate best performance per dataset.

4.2 Dataset-Specific Analysis

4.2.1 Iris Dataset Results

Model	Accuracy	F1-Score	AUROC
Custom Decision Tree	0.8667	0.8667	0.9000
Custom Random Forest	0.9111	0.9118	0.9844
Custom Extra Trees	0.9333	0.9333	0.9926
Sklearn Decision Tree	0.8667	0.8667	0.9000
Sklearn Random Forest	0.9556	0.9556	0.9926
Sklearn Extra Trees	0.9333	0.9333	0.9926

Table 3: Performance on Iris dataset. Best values per metric are in bold.

Observations:

- Sklearn Random Forest achieved the best overall performance with accuracy of 0.9556
- Custom implementations show progressive improvement: DT (0.8667) → RF (0.9111) → ET (0.9333)
- Ensemble methods significantly outperformed single decision trees (6.67% improvement for Extra Trees)
- Both custom and sklearn Extra Trees achieved identical AUROC (0.9926), demonstrating implementation correctness
- Custom and sklearn Decision Trees produced identical results (0.8667 accuracy), validating the base implementation

4.2.2 Wine Dataset Results

Model	Accuracy	F1-Score	AUROC
Custom Decision Tree	0.8889	0.8882	0.9221
Custom Random Forest	0.9815	0.9811	0.9998
Custom Extra Trees	0.9815	0.9811	1.0000
Sklearn Decision Tree	0.9074	0.9066	0.9346
Sklearn Random Forest	0.9630	0.9625	1.0000
Sklearn Extra Trees	0.9815	0.9811	1.0000

Table 4: Performance on Wine dataset. Best values per metric are in bold.

Observations:

- Custom Random Forest, Custom Extra Trees, and Sklearn Extra Trees achieved highest accuracy (0.9815)
- Perfect AUROC (1.0000) achieved by Custom Extra Trees, Sklearn Random Forest, and Sklearn Extra Trees
- Custom Random Forest outperformed Sklearn Random Forest in accuracy (0.9815 vs 0.9630)
- Ensemble methods showed substantial improvement over decision trees (9.26% for both RF and ET)
- Custom Extra Trees achieved perfect probabilistic predictions (AUROC = 1.0000)

5 Analysis and Discussion

5.1 Custom vs. Scikit-learn Implementations

Validation of Correctness: The custom implementations achieved identical or superior performance to scikit-learn in several cases:

- Decision Trees: Identical results on Iris (0.8667 accuracy)
- Extra Trees: Identical results on both datasets (Iris: 0.9333, Wine: 0.9815)
- Random Forest: Custom outperformed sklearn on Wine (0.9815 vs 0.9630)

This validates the correctness of the custom implementations.

Performance Differences: Key performance comparisons:

- On Iris, sklearn Random Forest achieved best accuracy (0.9556), outperforming custom RF (0.9111) by 4.44%
- On Wine, custom Random Forest excelled with 0.9815 accuracy, surpassing sklearn RF (0.9630) by 1.85%

- Custom Extra Trees matched sklearn Extra Trees exactly on both datasets
- Differences can be attributed to random seed effects, bootstrap sampling variations, and split selection strategies

5.2 Decision Tree vs. Ensemble Methods

Variance Reduction: Ensemble methods consistently showed superior performance compared to single decision trees:

- **Iris:** Custom DT (0.8667) → Custom RF (0.9111, +4.44%) → Custom ET (0.9333, +6.67%)
- **Wine:** Custom DT (0.8889) → Custom RF (0.9815, +9.26%) → Custom ET (0.9815, +9.26%)
- **AUROC improvement:** Iris DT (0.9000) → ET (0.9926); Wine DT (0.9221) → ET (1.0000)

This demonstrates effective variance reduction and improved generalization through ensemble aggregation.

Bias-Variance Tradeoff:

- **Decision Trees:** Lower bias (can fit complex patterns) but high variance (overfitting risk)
- **Random Forest:** Moderate bias, reduced variance through bagging and feature randomization
- **Extra Trees:** Slightly higher bias (random thresholds) but lowest variance

5.3 Random Forest vs. Extra Trees

Both methods showed strong performance with distinct characteristics:

Iris Dataset: Extra Trees outperformed Random Forest:

- Accuracy: ET (0.9333) vs RF (0.9111) - 2.22% improvement
- AUROC: ET (0.9926) vs RF (0.9844) - indicating better probabilistic predictions
- Additional randomness from random thresholds improved generalization

Wine Dataset: Both methods achieved identical accuracy (0.9815), but differed in AUROC:

- Extra Trees achieved perfect AUROC (1.0000)
- Random Forest achieved near-perfect AUROC (0.9998)
- Both demonstrated excellent separability of the Wine dataset

5.4 Dataset Characteristics

Wine Dataset:

- Higher dimensionality (13 features vs. 4)
- Larger training set (124 samples vs. 105)
- More separable classes (ensemble methods achieved 100%)
- Better suited for tree-based methods

Iris Dataset:

- Lower dimensionality challenges ensemble diversity
- Some class overlap (particularly Versicolor and Virginica)
- More challenging for achieving perfect classification

5.5 Computational Considerations

Training Complexity:

- Decision Tree: $O(n \cdot d \cdot \log n \cdot depth)$
- Random Forest: $O(T \cdot n \cdot m \cdot \log n \cdot depth)$ where $m = \sqrt{d}$
- Extra Trees: $O(T \cdot n \cdot m \cdot depth)$ (faster due to random thresholds)

Prediction Complexity:

- Decision Tree: $O(depth)$
- Random Forest/Extra Trees: $O(T \cdot depth)$

5.6 Strengths and Limitations

Strengths:

- Excellent performance on structured/tabular data
- No feature scaling required
- Handles non-linear relationships naturally
- Ensemble methods provide robust predictions
- Interpretability (especially for single trees)

Limitations:

- Risk of overfitting (single trees)
- Higher computational cost for ensembles
- Memory requirements increase with ensemble size
- Less effective on high-dimensional sparse data (e.g., text)

6 Conclusion

This assignment successfully implemented and evaluated three tree-based learning algorithms from scratch. Key findings include:

1. **Ensemble Superiority:** Random Forest and Extra Trees consistently outperformed single Decision Trees with improvements of 4.44-6.67% on Iris and 9.26% on Wine.
2. **Implementation Validation:** Custom implementations matched or exceeded scikit-learn performance. Notably, custom Random Forest outperformed sklearn on Wine (98.15% vs 96.30%), and Extra Trees achieved identical results.
3. **Variance Reduction:** Ensemble methods effectively reduced variance, with Extra Trees achieving perfect AUROC (1.0000) on Wine compared to Decision Tree's 0.9221.
4. **Dataset Characteristics:** Wine dataset proved more amenable to ensemble methods (98.15% accuracy) than Iris (95.56% best), likely due to higher dimensionality (13 vs 4 features).
5. **Extra Trees Advantage:** Extra Trees consistently achieved highest AUROC scores (Iris: 0.9926, Wine: 1.0000), demonstrating superior probabilistic predictions through random threshold selection.

6.1 Future Work

Potential extensions include:

- Implementing regression support
- Adding feature importance analysis
- Exploring different hyperparameter configurations
- Testing on larger and more complex datasets
- Implementing out-of-bag (OOB) error estimation for Random Forest
- Optimizing code for better computational performance

6.2 Lessons Learned

- Understanding algorithmic details is crucial for proper implementation
- Edge case handling (empty splits, single-class nodes) is essential for robustness
- Ensemble methods provide significant performance improvements (4-9% in our experiments)
- Proper evaluation requires multiple metrics (AUROC revealed Extra Trees' superiority despite similar accuracy)
- Custom implementations can match or exceed library performance when correctly designed
- Random seed selection affects results; reproducibility through fixed seeds is critical

References

- [1] Breiman, L. (2001). Random forests. *Machine learning*, 45(1), 5-32.
- [2] Geurts, P., Ernst, D., & Wehenkel, L. (2006). Extremely randomized trees. *Machine learning*, 63(1), 3-42.
- [3] Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning: data mining, inference, and prediction*. Springer Science & Business Media.
- [4] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of machine learning research*, 12(Oct), 2825-2830.